

Relazione progetti - Metodi del calcolo scientifico

Realizzazione di una libreria per risolvere sistemi lineari con
metodi iterativi e compressione di immagini tramite DCT

Autore: Marco Ferioli - matricola 879277

Data: 12 settembre 2025

Contents

1	Progetto 1 (Alternativo) - realizzazione di una libreria per risolvere sistemi lineari con metodi iterativi	2
1.1	Introduzione ed obiettivi	2
1.2	Struttura ed implementazione della libreria	2
1.3	Risultati ottenuti e considerazioni	4
1.4	Conclusioni	7

1 Progetto 1 (Alternativo) - realizzazione di una libreria per risolvere sistemi lineari con metodi iterativi

1.1 Introduzione ed obiettivi

L'obiettivo principale del progetto è stato quello di realizzare una libreria Python che implementasse alcuni metodi iterativi per risolvere sistemi lineari, andando successivamente a raccogliere dati relativi alle loro performance con lo scopo di effettuare confronti tra i vari metodi risolutivi. Sono stati implementati i seguenti metodi iterativi:

- Metodo di Jacobi;
- Metodo di Gauss-Seidel;
- Metodo del gradiente;
- Metodo del gradiente coniugato;

Tali metodi sono stati testati per risolvere sistemi lineari associati a quattro matrici differenti con quattro gradi di tolleranza.

1.2 Struttura ed implementazione della libreria

La libreria sviluppata utilizza NumPy per gestire le strutture dati e le operazioni elementari ed è stata implementata nel file `solvers.py`. Tutti i metodi iterativi presentano la medesima struttura di base, indicata nello pseudocodice seguente:

Algorithm 1: Procedura iterativa generica

Input: $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$, tol

Output: $x \in \mathbb{R}^n$

1. inizializzo $x^{(0)} \in \mathbb{R}^n$;
 2. $k = 0$;
 3. **while** *criterioDiArresto*($x^{(k)}$) **do**
 4. $x^{(k+1)} = \text{update}(x^{(k)})$;
 5. $k = k + 1$;
 6. **if** $k > \text{maxIter} = 20000$ **then**
 7. [errore]: il metodo non converge
 8. **break**;
 9. **end if**
 10. **end while**
 11. $x = x^{(k)}$;
-

Per questa ragione è stata sviluppata la funzione `_base_iterate_solver(A, b, x, tol, update_method)`, che implementa l'esatto algoritmo specificato in precedenza. La prima operazione messa in atto consiste nell'effettuare dei controlli preliminari sull'input fornito alla funzione. Infatti la matrice A associata al sistema lineare da risolvere deve essere quadrata; inoltre il vettore b , che rappresenta il termine noto, ed il vettore x , che rappresenta la soluzione esatta del sistema, devono avere la stessa dimensione della matrice A .

Il criterio di arresto selezionato per la procedura è il seguente:

$$\frac{\|Ax^{(k)} - \mathbf{b}\|}{\|\mathbf{b}\|} < \text{tol} \quad (1)$$

Durante lo svolgimento della funzione viene avviato un timer che calcola il tempo impiegato dal metodo iterativo per calcolare la soluzione approssimata x_h ; inoltre quest'ultima viene utilizzata per la computazione dell'errore relativo rispetto alla soluzione reale del sistema. L'errore relativo è stato calcolato con la formula:

$$e_{\text{rel}} = \frac{\|\mathbf{x}_h - \mathbf{x}\|}{\|\mathbf{x}\|}, \quad (2)$$

Il metodo di update viene invece passato come parametro della funzione; in questo modo è stato possibile evitare di ripetere l'intera procedura iterativa generica per ogni metodo. A questo punto sono state definite quattro funzioni, ciascuna delle quali definisce il proprio criterio di aggiornamento in relazione al metodo utilizzato:

- **Metodo di Jacobi**, implementato nella funzione `jacobi_solver(A, b, x, tol)`. Inizialmente viene effettuato un controllo sulla dominanza diagonale della matrice, perché rappresenta il criterio sufficiente di convergenza; successivamente è importante verificare che sulla diagonale della matrice **A** non siano presenti elementi nulli, perché altrimenti non è possibile applicare il metodo. Successivamente vengono calcolate le matrici **P** (estraendo la matrice diagonale di **A**) e P^{-1} . A questo punto viene definita una nuova funzione, che rappresenta il criterio di aggiornamento e viene effettuata una chiamata a `_base_iterate_solver`, fornendo il criterio di aggiornamento definito.
- **Metodo di Gauss-Seidel**, implementato nella funzione `gauss_seidel_solver(A, b, x, tol)`. Anche in questo caso viene effettuato un controllo sulla dominanza diagonale della matrice, perché rappresenta il criterio sufficiente di convergenza del metodo. Quindi viene calcolata la matrice **P** estraendo la matrice triangolare inferiore di **A**. Il criterio di update del metodo di Gauss-Seidel presuppone la risoluzione di un sistema lineare tramite il metodo della sostituzione in avanti, quindi è stata implementata la funzione `_linear_solve_forward(L, b)` con lo scopo di gestire tale operazione. Successivamente viene effettuata la definizione del criterio di update e la chiamata a `_base_iterate_solver`.
- **Metodo del gradiente e variante coniugata**, implementati nelle funzioni `gradient_solver(A, b, x, tol)` e `conjugate_gradient_solver(A, b, x, tol)`. In entrambi i casi viene effettuato un controllo sulla simmetria della matrice **A**, che inoltre deve essere definita positiva (deve soddisfare la decomposizione di Cholesky). Successivamente vengono definiti i rispettivi metodi di update e viene effettuata la chiamata a `_base_iterate_solver`.

Infine, il file `main.py` è un eseguibile che si occupa di caricare le matrici dai file forniti, eseguire i metodi iterativi per i diversi gradi di tolleranza, raccogliere i risultati e produrre i grafici.

1.3 Risultati ottenuti e considerazioni

I seguenti grafici e tabelle rappresentano il numero di iterazioni, l'errore relativo ed il tempo impiegato da ciascun metodo rispetto al grado di tolleranza indicato.

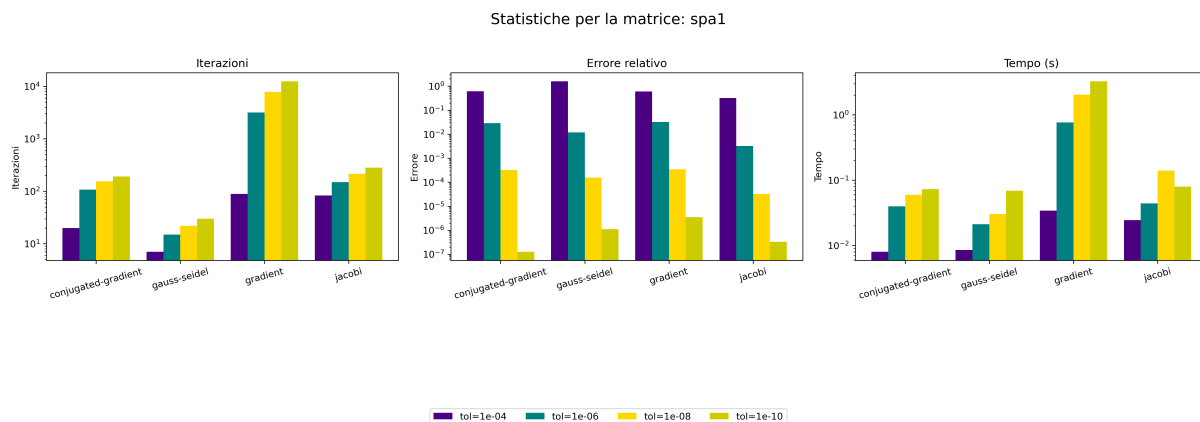


Table 1: Risultati per spa1.

Tolleranza	Metodo	Iterazioni	Tempo (s)	Errore relativo
1e-04	Gradiente Coniugato	20	0.00798	6.09e-01
	Gauss-Seidel	7	0.00851	1.61e+00
	Gradiente	89	0.03418	6.02e-01
	Jacobi	83	0.02444	3.17e-01
1e-06	Gradiente Coniugato	108	0.03959	2.91e-02
	Gauss-Seidel	15	0.02122	1.19e-02
	Gradiente	3175	0.76665	3.21e-02
	Jacobi	149	0.04411	3.22e-03
1e-08	Gradiente Coniugato	156	0.05969	3.26e-04
	Gauss-Seidel	22	0.03026	1.57e-04
	Gradiente	7823	2.03302	3.48e-04
	Jacobi	215	0.13986	3.26e-05
1e-10	Gradiente Coniugato	191	0.07301	1.26e-07
	Gauss-Seidel	30	0.06882	1.11e-06
	Gradiente	12509	3.27492	3.53e-06
	Jacobi	281	0.07983	3.31e-07

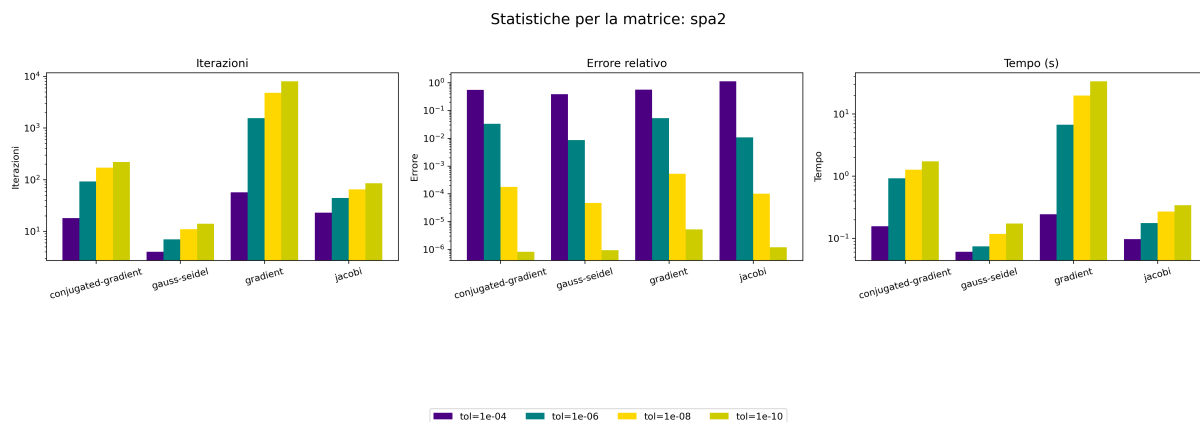


Table 1: Risultati per spa2.

Tolleranza	Metodo	Iterazioni	Tempo (s)	Errore relativo
1e-04	Gradiente Coniugato	18	0.15674	5.47e-01
	Gauss-Seidel	4	0.06046	3.86e-01
	Gradiente	57	0.24268	5.64e-01
	Jacobi	23	0.09667	1.13e+00
1e-06	Gradiente Coniugato	92	0.92354	3.34e-02
	Gauss-Seidel	7	0.07439	8.57e-03
	Gradiente	1543	6.72096	5.24e-02
	Jacobi	44	0.17591	1.07e-02
1e-08	Gradiente Coniugato	171	1.27186	1.78e-04
	Gauss-Seidel	11	0.11727	4.65e-05
	Gradiente	4793	19.82971	5.21e-04
	Jacobi	65	0.26926	1.01e-04
1e-10	Gradiente Coniugato	220	1.72761	8.07e-07
	Gauss-Seidel	14	0.17263	9.27e-07
	Gradiente	8031	33.60231	5.20e-06
	Jacobi	85	0.34098	1.19e-06

Statistiche per la matrice: vem1

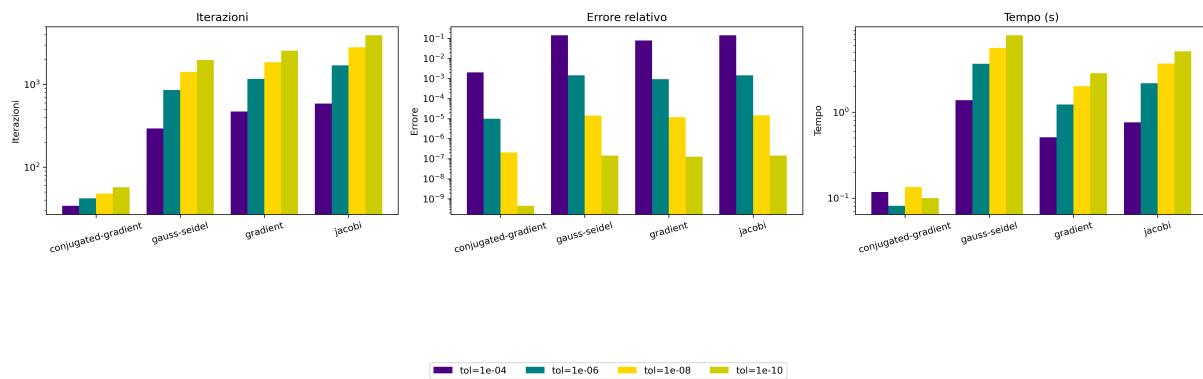


Table 1: Risultati per vem1.

Tolleranza	Metodo	Iterazioni	Tempo (s)	Errore relativo
1e-04	Gradiente Coniugato	34	0.11738	2.05e-03
	Gauss-Seidel	294	1.37918	1.45e-01
	Gradiente	472	0.50953	7.95e-02
	Jacobi	586	0.76134	1.45e-01
1e-06	Gradiente Coniugato	42	0.08113	9.86e-06
	Gauss-Seidel	854	3.66866	1.44e-03
	Gradiente	1170	1.22809	9.28e-04
	Jacobi	1705	2.17067	1.45e-03
1e-08	Gradiente Coniugato	48	0.13429	2.09e-07
	Gauss-Seidel	1414	5.60127	1.44e-05
	Gradiente	1856	2.00268	1.17e-05
	Jacobi	2824	3.69228	1.45e-05
1e-10	Gradiente Coniugato	57	0.10006	4.45e-10
	Gauss-Seidel	1974	7.87696	1.44e-07
	Gradiente	2570	2.82954	1.24e-07
	Jacobi	3943	5.09417	1.45e-07

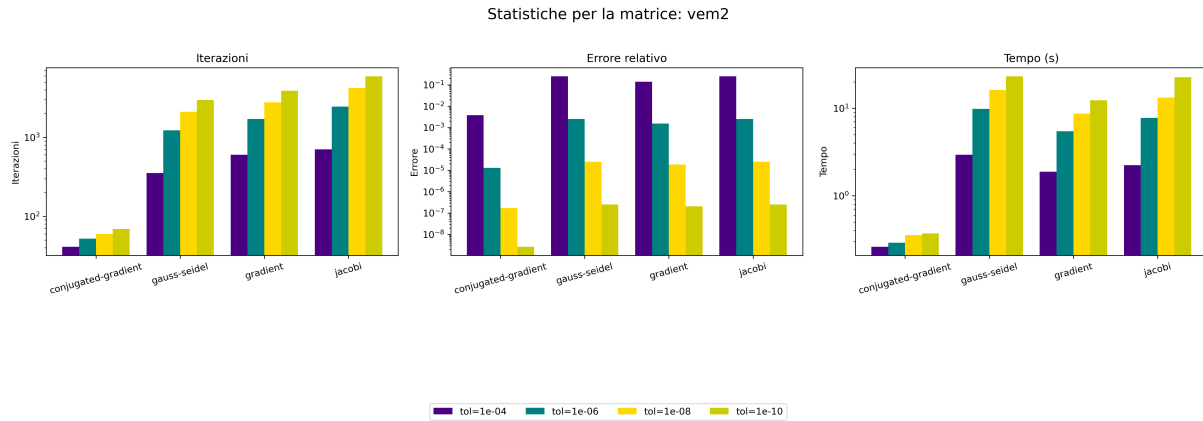


Table 1: Risultati per vem2.

Tolleranza	Metodo	Iterazioni	Tempo (s)	Errore relativo
1e-04	Gradiente Coniugato	41	0.26033	3.76e-03
	Gauss-Seidel	353	2.93487	2.53e-01
	Gradiente	602	1.88137	1.39e-01
	Jacobi	704	2.23762	2.54e-01
1e-06	Gradiente Coniugato	52	0.28995	1.31e-05
	Gauss-Seidel	1228	9.85558	2.53e-03
	Gradiente	1706	5.46532	1.53e-03
	Jacobi	2454	7.74447	2.53e-03
1e-08	Gradiente Coniugato	60	0.35604	1.72e-07
	Gauss-Seidel	2103	16.26478	2.52e-05
	Gradiente	2786	8.69872	1.87e-05
	Jacobi	4203	13.26090	2.53e-05
1e-10	Gradiente Coniugato	69	0.37114	2.61e-09
	Gauss-Seidel	2978	23.25267	2.52e-07
	Gradiente	3892	12.30749	2.06e-07
	Jacobi	5951	22.70304	2.54e-07

Sulla base dei risultati ottenuti è possibile effettuare le seguenti considerazioni:

- Tutte le matrici fornite come test rispettano i criteri di simmetria e sono definite positive; tuttavia non hanno dominanza diagonale. Nonostante ciò il metodo del gradiente e di Gauss-Seidel convergono nelle 20000 iterazioni predisposte, così come i metodi del gradiente e la variante coniugata, confermando quanto descritto dal punto di vista teorico.
- In tutti i casi si può notare che l'errore relativo decresce con il diminuire della tolleranza; come diretta conseguenza, aumentano anche il numero di iterazioni necessarie per arrivare alla convergenza. Questo comportamento è consistente dal punto di vista teorico; inoltre per tutte le matrici si raggiunge, con la minima tolleranza, un errore relativo accettabile (compreso generalmente tra $5.20e^{-6}$ e $4.45e^{-10}$).
- Il tempo cresce all'aumentare del numero di iterazioni e al diminuire della tolleranza; anche questo comportamento è coerente dal punto di vista teorico. In alcuni casi è possibile notare del rumore nel tempo di esecuzione (ad esempio si ha un tempo leggermente minore per livelli di tolleranza più bassi), probabilmente dovuto a fattori esterni (CPU, garbage collector, threads utilizzati).
- Il metodo del gradiente presenta un alto numero di iterazioni per tutte le matrici prese in considerazione; inoltre non raggiunge un errore relativo più basso rispetto

agli altri metodi. Probabilmente la causa di questo comportamento è la natura stessa del metodo risolutivo; in particolare la discesa viene spesso effettuata con un'andatura "a zig-zag". Infatti il metodo del gradiente coniugato converge sempre in meno iterazioni; ciò è coerente dal punto di vista teorico, perché tale metodo presenta l'ottimizzazione sulla direzione di discesa.

- È stata inoltre condotta una semplice analisi sugli elementi nulli di ciascuna matrice; i risultati ottenuti indicano che le matrici **spa1** ed **spa2** presentano tra l'81.7% e l'81.87% di elementi nulli; al contrario le matrici **vem1** e **vem2** contengono tra il 99.53% ed il 99.7% di elementi nulli. Tutte le matrici sono sparse, ma **vem1** e **vem2** presentano una percentuale di elementi nulli molto più alta. Questa considerazione è compatibile con il fatto che l'algoritmo del gradiente coniugato risulta essere quello che compie meno iterazioni (ed ha tempo di computazione minimo) per raggiungere la convergenza per tali matrici, segnalando che tale algoritmo è più performante su matrici con un alto numero di elementi nulli.
- Sono stati calcolati anche i valori di condizionamento delle varie matrici, tramite il comando `cond()` della libreria `NumPy`. I risultati indicano che le matrici **spa1** e **spa2** presentano un valore di condizionamento alto, rispettivamente 1000 e 2000; al contrario le matrici **vem1** e **vem2** hanno valore di condizionamento pari a 300 e 500. Ciò si traduce nel fatto che le matrici **spa1** ed **spa2** hanno soluzioni che risentono maggiormente delle approssimazioni dovute all'aritmetica floating point, infatti l'errore relativo risulta essere più alto.

1.4 Conclusioni

I metodi di Jacobi e Gauss-Seidel risultano funzionare meglio sulle matrici **spa1** ed **spa2** e rappresentano un punto di incontro tra la semplicità di implementazione e le prestazioni, il metodo del gradiente ha delle prestazioni non eccezionali sulle matrici proposte a causa della procedura di aggiornamento, mentre il metodo del gradiente coniugato rappresenta un buon compromesso tra tutti i criteri risolutivi selezionati. Quest'ultimo, in particolare funziona meglio su matrici sparse, perché le più comuni librerie (come `NumPy`) permettono di ottimizzare le operazioni base tra matrici e vettori sfruttando la struttura sparsa della matrice.